

Edu Platform — Dashboard Requirements

Overview

The Edu Platform has three main user roles:

- **Admin**
- **Instructor**
- **Student**

Each role must have a **different dashboard** based on their responsibilities.

The dashboard should not simply display generic statistics. It should answer:

"What does this user need to know or do right now?"

The frontend is built with **React**, while the backend is **Django / Django REST Framework**.

All dashboard data should come from Django APIs.

Do not hardcode dashboard statistics.

1. Student Dashboard

Purpose

The Student Dashboard is focused on:

- Learning progress
- Courses
- Assignments
- Quizzes
- Grades
- Upcoming deadlines
- Notifications
- Career preparation

The student should immediately understand:

What am I learning? What do I need to do? How am I performing? How close am I to being career-ready?

1.1 Welcome Section

Display:

Welcome back, {Student Name} 🙌
Continue your learning journey and build your skills.

Optionally display the current date.

1.2 Summary Cards

Display four main statistics:

Enrolled Courses

Number of courses the student is currently enrolled in.

Example:

12
Enrolled Courses

Overall Progress

Average progress across enrolled courses.

Example:

68%
Overall Progress

Pending Assignments

Number of assignments that have not yet been submitted.

Example:

4
Pending Assignments

Average Score

Student's overall assessment average.

Example:

84%
Average Score

These values must be retrieved from Django APIs.

2. Continue Learning

This should be one of the most prominent sections.

Display the student's currently active courses.

Each course card should contain:

- Course name
- Instructor
- Course image/thumbnail if available
- Progress percentage
- Progress bar
- Last completed lesson
- Next lesson
- Continue Learning button

Example:

Python Programming
Instructor: Ahmed Mohamed

Progress
 75%

Last lesson:
Functions and Modules

Next:
Object-Oriented Programming

[Continue Learning]

The Continue button should navigate the student to the appropriate lesson/course page.

3. My Courses

Display the student's enrolled courses.

Each course should show:

- Course name
- Instructor
- Progress
- Course status
- Number of completed lessons
- Total lessons
- Continue/View Course button

Possible statuses:

In Progress
Completed
Not Started

Provide a:

View All Courses

button.

4. Upcoming Deadlines

Display assignments and quizzes that require student action.

Each item should contain:

- Assignment/quiz name
- Course name
- Due date

- Remaining time
- Submission status
- Action button

Example:

Python Assignment #3
Python Programming

Due:
Aug 15, 2026

Status:
Pending

[View Assignment]

Sort items by nearest deadline first.

5. Recent Grades

Display recently graded assessments.

Each item should contain:

- Assessment name
- Course
- Score
- Maximum score
- Percentage
- Date

Example:

SQL Quiz #2
Database Fundamentals

18 / 20
90%

Aug 12, 2026

Provide:

View All Grades

if the platform has a dedicated grades page.

6. Learning Progress

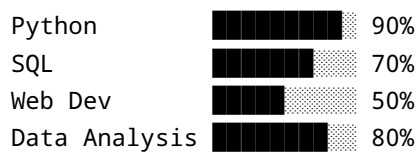
Display a visual representation of student progress.

Possible visualization:

- Course progress
- Weekly learning activity
- Completed lessons
- Assessment performance

Example:

Course Progress



Charts should be simple and readable.

Use a chart library only if one is already installed in the project.

Do not introduce a large new dependency unless necessary.

7. Notifications

Display recent notifications such as:

- New assignment
- New quiz
- Grade published
- New course material
- Instructor announcement
- Deadline reminder

Example:

 **New Assignment**
Python Assignment #4 was published.

2 hours ago

Display the most recent notifications first.

Provide:

[View All Notifications](#)

if a notification page exists.

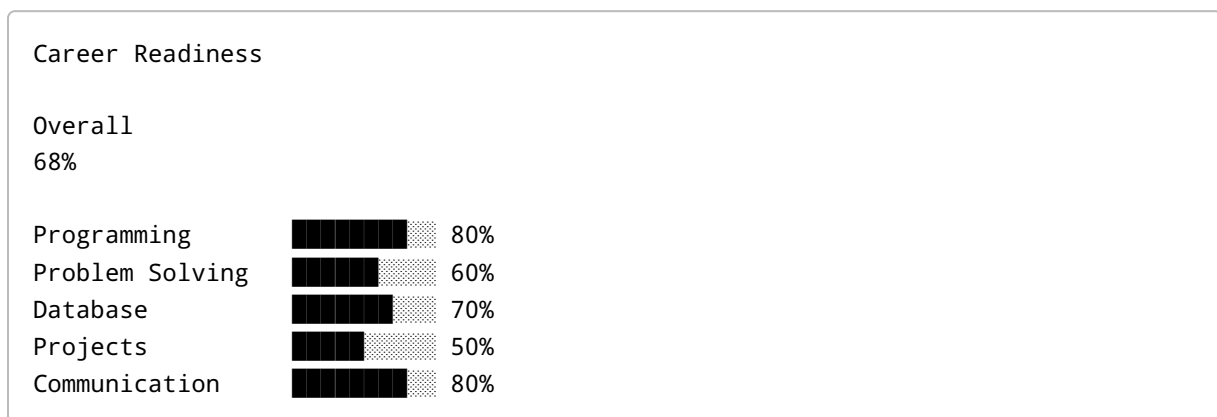
8. Career Readiness

This is an important feature for Daltex School.

The goal of the platform is not only to teach courses but to prepare students for entering the technology/ job market.

Display a **Career Readiness** section.

Example:



The exact calculation should eventually come from Django based on:

- Course completion
- Quiz performance

- Assignment performance
- Projects
- Instructor evaluations
- Other relevant assessments

If the backend does not currently provide this data, create the UI in a way that can easily consume it later.

Do not invent fake backend data.

9. Student Quick Actions

Provide convenient actions:

Browse Courses

My Courses

View Assignments

View Quizzes

View Progress

Open Wallet

View Notifications

Only display actions that actually exist in the current application.

10. Instructor Dashboard

Purpose

The Instructor Dashboard focuses on:

- Courses
- Students
- Assignments
- Quizzes
- Submissions
- Grades
- Student performance

The instructor should immediately know:

What do I need to review? How are my students performing? What courses am I managing?

11. Instructor Summary Cards

Display:

My Courses

Number of courses assigned to the instructor.

Total Students

Number of students enrolled in the instructor's courses.

Pending Submissions

Number of assignments waiting for grading.

Average Student Score

Average performance of students in the instructor's courses.

Example:

My Courses	5
Total Students	142
Pending Reviews	18
Average Score	81%

12. Instructor Courses

Display courses belonging to the instructor.

Each course should show:

- Course name
- Number of students
- Course progress/activity
- Number of lessons
- Number of assignments
- Number of quizzes
- Manage Course button

Example:

Python Programming

Students: 42

Lessons: 18

Assignments: 6

Quizzes: 4

[Manage Course]

An instructor must only see/manage courses they are authorized to access.

13. Pending Submissions

This should be a major section of the instructor dashboard.

Display assignments that require grading.

Each item:

- Assignment name
- Course
- Student
- Submission date
- Status
- Review button

Example:

Python Assignment #3

Python Programming

12 submissions waiting for review

[Review Submissions]

Sort by oldest pending submission first.

14. Student Performance

Display performance information for the instructor's students.

Possible metrics:

- Average score
- Course completion
- Assignment performance
- Quiz performance
- Students needing attention

Example:

Student Performance

Average Score	81%
Completion Rate	74%

Students needing attention:

- Ahmed
- Sara
- Omar

The instructor must only see students enrolled in their own courses.

15. Upcoming Deadlines

Display:

- Assignment deadlines
- Quiz dates
- Course milestones

Example:

Upcoming

Python Assignment #4
Due Aug 15

SQL Quiz #3
Aug 17

16. Recent Activity

Display recent course activity.

Examples:

```
Ahmed submitted Python Assignment #3  
Sara completed SQL Quiz #2  
Omar enrolled in Web Development  
New assignment created
```

Sort by newest activity first.

17. Instructor Quick Actions

Provide:

```
Create Course  
Add Lesson  
Create Assignment  
Create Quiz  
Review Submissions  
View Students
```

Only show actions that the instructor has permission to perform.

18. Admin Dashboard

Purpose

The Admin Dashboard provides a high-level overview of the entire platform.

The Admin should immediately understand:

How many users are on the platform? How many courses exist? How active is the platform?
Are there problems requiring attention?

19. Admin Summary Cards

Display:

Total Students

Number of registered students.

Total Instructors

Number of registered instructors.

Total Courses

Number of courses.

Total Enrollments

Total course enrollments.

Example:

Students	850
Instructors	42
Courses	38
Enrollments	2,450

20. Platform Overview

Display useful platform statistics.

Possible charts:

User Registration

Show new users over time.

Students registered per month

Course Enrollment

Show course enrollment trends.

Course Completion

Show completion percentage.

Student Performance

Show overall assessment performance.

Use charts only where they provide meaningful information.

Avoid filling the dashboard with unnecessary graphs.

21. User Statistics

Display:

Students	850
Instructors	42
Admins	3

Optionally show:

- Active users
 - Inactive users
 - New registrations
-

22. Course Statistics

Display useful course information:

- Most popular courses
- Highest enrollment
- Lowest enrollment
- Highest completion rate
- Lowest completion rate
- Active courses
- Inactive courses

Example:

Most Popular Courses

1. Python Programming	250 students
2. Web Development	210 students
3. Data Analysis	185 students

23. Recent Registrations

Display recently registered users.

Each row:

- Name
- Role
- Registration date
- Account status

Example:

Ahmed Mohamed	Student	Aug 12	Active
Sara Ali	Student	Aug 11	Active
Omar Hassan	Instructor	Aug 10	Active

Provide:

[View All Users](#)

24. Recent Platform Activity

Display important recent actions.

Examples:

New student registered
New instructor added
New course created

```
Student enrolled in course
Assignment submitted
Course updated
```

This should eventually come from a Django activity/audit API.

25. System Alerts

Show things that require Admin attention.

Examples:

```
△ Course without instructor
△ Inactive instructor
△ Course with unusually low enrollment
△ Failed payment
△ System issue
```

Only implement alerts that are supported by the backend.

Do not create fake alerts.

26. Admin Quick Actions

Provide:

```
Add Student
Add Instructor
Create Course
Manage Users
Manage Courses
Manage Enrollments
View Statistics
```

27. Dashboard Layout

The dashboard should follow a clean structure.

Recommended general layout:

Welcome / Page Header			
Statistic	Statistic	Statistic	Statistic
Main Content		Upcoming / Actions	
Charts / Performance		Notifications	

Do not force every dashboard to use exactly the same layout.

The content should reflect the role.

28. Responsive Design

All dashboards must work on:

- Desktop
- Laptop
- Tablet
- Mobile

The dashboard should not overflow horizontally on small screens.

Cards should automatically adapt:

Desktop:
4 cards per row

Tablet:
2 cards per row

Mobile:
1 card per row

29. Dark Mode

The project already supports dark mode.

Dashboard components must support both:

```
Light Mode
Dark Mode
```

Use the existing Tailwind dark-mode classes and project styling.

Do not create a separate dark-mode implementation.

30. Backend Integration

Dashboard information must come from Django APIs.

Do not hardcode:

```
850 students
42 instructors
68% progress
```

These are examples only.

The actual values must be fetched from Django.

Suggested conceptual API structure:

```
/api/dashboard/student/
/api/dashboard/instructor/
/api/dashboard/admin/
```

The exact endpoints should be determined by inspecting the existing Django backend.

Do not create duplicate endpoints if equivalent APIs already exist.

31. Loading States

Every dashboard section that depends on an API should have a loading state.

Example:

Loading...

Prefer skeleton loaders if the project already uses them.

Do not show incorrect values while data is loading.

32. Error States

If a dashboard API fails, display a user-friendly error.

Example:

Unable to load dashboard data.

[Try Again]

Do not crash the entire dashboard because one API request failed.

Where practical, sections should fail independently.

For example:

Courses → Loaded
Assignments → Loaded
Notifications → Failed

The entire page should still work.

33. Empty States

Handle cases where the user has no data.

Student

You haven't enrolled in any courses yet.

[[Browse Courses](#)]

Instructor

You don't have any courses assigned yet.

Admin

No recent activity.

Avoid showing empty charts or broken components.

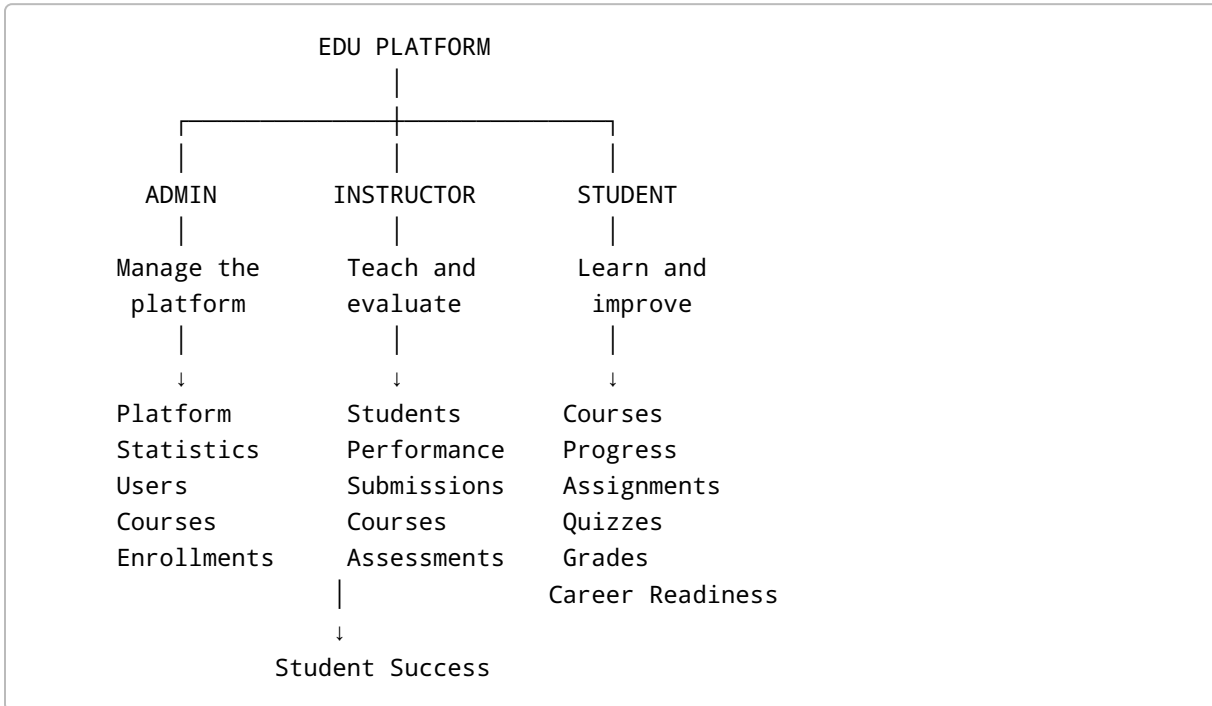
34. Important Development Rules

When implementing the dashboards:

1. Inspect the existing React project before creating new components.
 2. Inspect the existing Django APIs before creating API calls.
 3. Reuse existing components and styling.
 4. Reuse the existing AuthContext.
 5. Reuse existing routing/protected routes.
 6. Do not hardcode dashboard statistics.
 7. Do not create fake API responses.
 8. Do not invent backend endpoints without checking the Django project.
 9. Keep API logic separate from UI components where practical.
 10. Handle loading, error, and empty states.
 11. Make the dashboard responsive.
 12. Support the existing dark mode.
 13. Follow the existing project's naming and folder conventions.
 14. Do not introduce unnecessary dependencies.
 15. Do not modify authentication unless specifically required.
 16. Django remains the source of truth for authorization.
 17. Frontend role checks are for UI/UX and routing only.
 18. Never expose another user's private data.
-

35. Final Dashboard Goal

The three dashboards should have clearly different purposes:



The dashboard should prioritize **actionable information**, not just statistics.

A good dashboard should allow a user to understand their current situation within a few seconds and quickly navigate to the action they need to take.